

Guised Up

Take-Home Technical Project

Full-Stack Developer Assessment

Before you start, read this.

This project is designed to reveal how you think, not just what you know. There are intentionally open-ended sections. A senior engineer will read your technical solution doc as carefully as your code. Don't just build — explain your decisions.

Time: 1 full day (8 hours). You are expected to use AI agentic tools. Not using them will be a red flag.

Field	Details
Stack	Full-Stack Developer
Time Limit	React Native • Laravel PHP • Python • SQL • Vector DB
Submission	1 day
AI Tools	GitHub repo + solution doc (PDF or Notion link)
Contact	Required — Cursor, Copilot, Claude, etc.
	DM founder on LinkedIn/ whatsapp the number from where you received this if you have blockers

01. The Context

Guised Up is a social platform being built to help people show up authentically online — no curated highlight reels, no follower-count anxiety. Real people, real connections.

You're joining as a founding engineer. This isn't a CRUD exercise. We want to see how you think when given a real product problem.

The app is a React Native mobile application with a Laravel PHP + Python backend, SQL databases, and vector DB for intelligent search and recommendations. You are expected to work fast — using AI agentic tools to reach 80%+ efficiency.

02. The Product Brief

Read this carefully. Your technical solution doc will be evaluated on how well you comprehend and respond to this brief.

Feature: Real Connections Feed

Guised Up needs a personalized "RealConnections" feed. Unlike Instagram or Twitter, this feed should NOT rank content by engagement metrics (likes, shares, comments). Instead, it should rank content by:

- Authenticity signals — posts with fewer filters, less-polished images, and genuine text get ranked higher
- Relationship depth — content from people you interact with genuinely (not just follow) surfaces first
- Semantic similarity — the feed understands what topics you care about using vector embeddings
- Time decay — newer content is preferred, but not at the expense of relevance

Users can also search the feed using natural language. For example: "funny travel stories from last week" should return semantically relevant posts — not keyword matches.

03. What You Need to Build

Complete all 4 parts. Each part tests a different dimension of your skill.

Part A — Technical Solution Document

Before writing a single line of code, produce a Technical Solution Document (TSD) for the Real Connections Feed feature described above.

Your TSD must include:

- System architecture diagram (even a rough ASCII/diagram.io sketch is fine)
- Database schema design — tables, relationships, indexes
- How you plan to implement vector embeddings and which vector DB you'd use and why
- API design — endpoints, request/response shapes, auth strategy
- Feed ranking algorithm — explain the logic in plain English first, then in pseudocode
- What AI agentic tools you used and how they helped you (be honest — this matters)
- Any trade-offs or assumptions you made

Format: PDF, Notion page, or Markdown in your repo. Length: as long as it needs to be, not a word more.

Part B — Backend API (Laravel PHP + Python)

Build the following API endpoints. Python can be used for ML/embedding tasks; Laravel for the main API layer.

Endpoint	Description
POST /api/posts	Create a new post. Accept text + optional image URL. Auto-generate a vector embedding for the post content and store it.
GET /api/feed	Return a personalized feed for the authenticated user. Use your ranking logic from the TSD. Return paginated results (20 per page).
GET /api/search?q={query}	Natural language search across posts using vector similarity. Return top 10 semantically relevant results.
POST /api/interactions	Log a user interaction (view, reply, reaction) against a post. This feeds the relationship-depth signal.

Requirements:

- Auth: UseLaravel Sanctum (token-based). Seed at least 2 test users.
- SQL: UseMySQL or PostgreSQL. Migrations must be included.
- Vector DB:Useany — Pinecone, Weaviate, pgvector, Qdrant, or Chroma. Explain your choice intheTSD.
- Embeddings:Use any open model (OpenAI, sentence-transformers, etc.) or mock it with a simple hashifyou don't have API credits — just explain the swap.
- Tests: Writeatleast 3 unit or feature tests for your most critical logic.

Part C — React Native Screen

Build a single React Native screen: the Feed Screen.

The Feed Screen must:

- Fetch and displaythe paginated feed from your GET /api/feed endpoint
- Each post cardshould show: avatar placeholder, username, post text, time ago, and a reaction button
- Implement infinitiescroll (load next page when user reaches bottom)
- Include a searchbar at the top that calls GET /api/search and shows results inline
- Handle loading,empty, and error states gracefully

UI quality matters. It doesn't need to be pixel-perfect, but it should feel intentional. No default React Native styles.

Part D — SQL Challenge

Write raw SQL queries for the following. Include them in a /sql folder in your repo.

D1	Query Required Return the top 10 most active users in the last 7 days, ranked by total interactions (views + replies + reactions).
D2	For a given user_id, return all posts from users they interact with most, ordered by interaction frequency descending, limited to posts from the last 30 days.
D3	Find any posts that have been viewed more than 100 times but have zero reactions. Return post_id, author_id, view_count, and created_at.
D4	Write a query that would help detect potential spam — users who have created more than 20 posts in the last 24 hours. Include their email and post count.

04. How You’ll Be Evaluated

We score every submission across 5 dimensions. The TSD carries as much weight as the code.

MAKE A VIDEO OF THE END PRODUCT EXPLAINING THE FEATURES

		What We Look For
Dimension Technical Solution Doc	Weight 25%	Does it show deep comprehension of the brief? Are decisions explained clearly?
Backend Quality	25%	Clean architecture, correct SQL schema, working vector search, tests present
React Native Screen	20%	Functional, well-structured, handles edge cases, looks intentional
SQL Queries	15%	Correctness, efficiency, readability — we'll run them against a real DB
AI Tool Usage	15%	Evidence of smart AI-augmented workflow. We want to see you move fast.

Instant disqualifiers

- No TechnicalSolution Document submitted
- Copy-pastedboilerplate with no evidence of understanding
- No migrationsor the DB schema is not reproducible
- NO EXPLANATION VIDEO

- Plagiarised work — we will check

05. How to Submit

Step	Action
1	Create a private GitHub repo named Guised Up-assessment-[yourname]
2	Push your full code with a clear README (setup instructions, how to run, .env.example)
3	Include your TSD in the repo as /docs/TSD.pdf, /docs/TSD.md, or link a Notion page in your README Include your SQL queries in /sql/queries.sql
4	Send the GitHub repo link + your TSD link to the founder via LinkedIn DM or the email you received this from
5	Subject line: [Guised Up Application] Your Name — Full-Stack Assessment
6	Make a video of the app that you build with explanation about features

06. Rules & Notes

- You may use any libraries, frameworks, or tools you normally would.
- You MUST use AI agentic tools (Cursor, GitHub Copilot, Claude, etc.). Document which ones and how in your TSD.
- You may ask the founder ONE clarifying question via LinkedIn DM. Use it wisely.
- Do not share this project with anyone else. It is confidential.
- Partial submissions are acceptable. Submit what you have and explain what you ran out of time for in your README.
- If you cannot complete the vector DB integration due to API credits, mock it clearly and explain the real implementation in your TSD.

YOU MUST MAKE THE VIDEO WITH EXPLANATION

A note from the founder

We designed this project to feel like a real day at Guised Up. You're given a product brief, expected to think through the architecture, write clean code fast, and explain your decisions clearly. That's the job.

We don't expect perfection. We expect hunger, clarity, and evidence that you used every tool available to you to move fast and build well. Good luck — and if you came here with a point to prove, build something from India to the world, now's your moment.